

Universidad de Caldas
Facultad de Inteligencia Artificial e Ingenierías
Análisis y Diseño de Algoritmos — 2026-1

Proyecto K-QGMIP

Manual de Usuario

Análisis de k -particiones de mínima información
Estrategias **KGeoMIP** y **KQNodes**

Guía de instalación y uso para usuarios finales

8 de junio de 2026

Índice

1. Introducción y visión general	2
1.1. ¿Qué hace el software?	2
1.2. ¿Para qué sirve?	2
1.3. Conceptos básicos	2
1.4. Capacidades y limitaciones	2
2. Requisitos del sistema	2
3. Instalación paso a paso	3
4. Video tutorial de instalación y uso	3
5. Guía de uso básico	4
5.1. Preparación de los datos de entrada	4
5.2. Ejecución básica	4
5.3. Interpretación de resultados	4
5.4. Casos de uso típicos	5
6. Opciones y parámetros avanzados	5
6.1. Parámetros de configuración	5
6.2. Modos de operación	5
6.3. Opciones de salida y visualización	6
6.4. Optimización de rendimiento	6
7. Interfaz web (sin línea de comandos)	6
8. Solución de problemas	7
9. Ejemplos y tutoriales	7
9.1. Tutorial básico (4 nodos)	7
9.2. Caso intermedio (10 nodos, varios k)	7
9.3. Ejemplo avanzado (rejilla completa)	7
10. Referencia rápida	8
10.1. Comandos principales	8
10.2. Glosario	8
10.3. Formato de archivos	8

1. Introducción y visión general

1.1. ¿Qué hace el software?

K-QGMIP analiza un **sistema** de variables binarias (descrito por una matriz de transición) y encuentra la mejor manera de **dividirlo en k partes** perdiendo la menor cantidad de información posible. En pocas palabras: responde a la pregunta “*si tuviera que romper este sistema en k pedazos, ¿por dónde lo cortaría para que el corte “duela” lo menos posible?*”.

1.2. ¿Para qué sirve?

Es una herramienta de **Teoría de la Información Integrada (IIT)**, usada para medir cuán “integrado” está un sistema (cuánto depende el todo de sus partes). Aplicaciones típicas: análisis de redes neuronales o genéticas, estudio de modularidad de sistemas dinámicos, y experimentación académica sobre IIT.

1.3. Conceptos básicos

- **Sistema:** conjunto de n nodos que pueden estar en 0 o 1.
- **k -partición:** una división del sistema en exactamente k grupos (bloques). Con $k = 2$ es un solo corte; con $k = 3, 4, 5$ son cortes múltiples.
- **Partición de mínima información (MIP):** la división que menos información destruye. El número que mide ese “daño” se llama **pérdida** (símbolo ϕ o δ_k): *cuanto más pequeño, mejor partición.*

Idea clave: el programa prueba muchos cortes posibles y se queda con el que produce la *pérdida más baja*. Ese es el resultado que usted busca.

1.4. Capacidades y limitaciones

- **Sí puede:** analizar sistemas pequeños y medianos ($n \leq 15\text{--}20$ nodos) con las estrategias exactas y rápidas KGeoMIP/KQNodes, para cualquier $k \in \{2, 3, 4, 5\}$.
- **Con cuidado:** sistemas grandes ($n \approx 25$) solo con la estrategia de *clustering*, que *propone* una división rápida pero no calcula la pérdida exacta (requiere demasiada memoria).
- **No puede:** garantizar la partición óptima global en sistemas grandes (usa heurísticas), ni manejar $n \gg 25$ en un equipo común.

2. Requisitos del sistema

Tabla 1: Requisitos mínimos y recomendados.

	Mínimo	Recomendado
Sistema operativo	Linux / macOS / Windows 10+	Linux (probado en Arch)
Python	3.14	3.14.5
RAM	4 GB ($n \leq 12$)	8–16 GB ($n \leq 20$); 8 GB+ para $n = 25$
Disco	500 MB	2 GB (incluye muestras grandes)
CPU	2 núcleos	4+ núcleos (paralelismo PCD)

Software necesario: el gestor de paquetes [uv](#) (instala Python y las dependencias automáticamente). Las bibliotecas (numpy, scipy, pandas, matplotlib, etc.) se declaran en `pyproject.toml` y se instalan con un solo comando.

3. Instalación paso a paso

Paso 1 — Obtener el código. Clonar el repositorio o descargar el `.zip`:

```
1 git clone <URL-del-repositorio> KQGMIP
2 cd KQGMIP
```

Paso 2 — Instalar uv (si aún no lo tiene):

```
1 curl -LsSf https://astral.sh/uv/install.sh | sh # Linux / macOS
2 # Windows (PowerShell):
3 # powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Paso 3 — Instalar dependencias. Desde la carpeta del proyecto:

```
1 uv sync
```

Esto crea el entorno virtual y descarga todas las bibliotecas. No es necesario activar el entorno: cada comando se ejecuta con `uv run`.

Paso 4 — Verificar la instalación.

```
1 uv run pytest -q # debe terminar con todos los tests en verde
2 uv run exec.py # ejecuta un análisis de ejemplo
```

Nota. Si `uv run pytest` pasa, el software quedó correctamente instalado. El primer `uv sync` puede tardar varios minutos según la red.

4. Video tutorial de instalación y uso

Se incluye un video demostrativo (formato MP4, 8–15 min) que cubre la instalación desde cero, la configuración y un ejemplo completo de ejecución e interpretación.

Video tutorial: [video/tutorial_KQGMIP.mp4](#)

(Enlace alternativo en línea: *pendiente de publicación*)

Contenido del video: (1) instalación de `uv` y `uv sync`; (2) edición de parámetros en `main.py`; (3) ejecución con distintos valores de k ; (4) interpretación de la salida; (5) generación de figuras.

5. Guía de uso básico

5.1. Preparación de los datos de entrada

El sistema se describe con una **TPM** (matriz de transición) en un archivo CSV: `data/samples/N{n}-{página}.csv`, con 2^n filas y n columnas de 0/1, en orden *little-endian* (la fila 0 es el estado 00...0). El programa la carga automáticamente según el número de nodos y la página.

Si no tiene una TPM, puede generar una desde la **línea de comandos** con el script dedicado (no interactivo con `-yes`):

```
1 uv run scripts/generate_tpm.py --n 4 # determinista 0/1 (N4?.csv)
2 uv run scripts/generate_tpm.py --n 6 --continuous # probabilidades continuas
3 uv run scripts/generate_tpm.py --n 10 --seed 7 --yes
```

El archivo se guarda en `data/samples/` con el siguiente sufijo libre (A, B, ...). También puede generarlo desde la **interfaz web** (§7), sin línea de comandos.

5.2. Ejecución básica

El análisis individual se configura en `main.py` y se lanza con `exec.py`. Los parámetros principales están al inicio de `main.py`:

```
1 initial_state = "1111011011" # Estado en t=0 (10 nodos)
2 conditions   = "1110001100" # 0 = condicionar (fijar) la variable
3 purview      = "0011101111" # 0 = marginalizar el nodo futuro
4 mechanism    = "1000111111" # 0 = marginalizar el nodo presente
5 k = 3        # Numero de particiones (k >= 2)
6
7 STRATEGY_NAME = "kgeomip" # kgeomip / kqnodes / clustering / exhaustive
8 PAGE = "A" # Pagina de red (A, B, C, ...)
```

Luego:

```
1 uv run exec.py
```

5.3. Interpretación de resultados

La salida muestra la estrategia usada, las distribuciones, la k -partición encontrada (bloques B1, B2, ...) y la **pérdida mínima** (ϕ). Ejemplo real (KGeoMIP, $k = 3$, red N10A):

```
1 KGeoMIP(k=3) fue la estrategia de solucion.
2 Distancia metrica: distancia-hamming
3 Notacion indexado: little-endian
4
5 Distribucion marginal del Subsistema:
6 [ 0.2500 0.5000 1.0000 ]
7 Distribucion marginal de la Particion:
8 [ 0.4062 0.4688 1.0000 ]
9
10 Mejor particion:
11 B1: | C
12 B2: | G
13 B3: agh | H
14 Perdida minima ( phi ) = 0.1875
15
16 Tiempos de ejecucion:
17 Horas: 0.00 = Minutos: 0.0 = Segundos: 0.0036
```

Cómo leerlo:

- **Bloques B1–B3:** cada bloque es futuro | presente. Las letras minúsculas son nodos de efecto ($t+1$); las mayúsculas, de mecanismo (t). Un lado vacío significa que ese bloque no aporta átomos de ese tiempo.
- **Pérdida mínima (ϕ):** 0,1875. Es el indicador clave: *más bajo es mejor*. Sirve para comparar estrategias o valores de k .
- **Tiempo:** cuánto tardó el análisis.

5.4. Casos de uso típicos

1. **Una 3-partición de un sistema pequeño:** ponga `STRATEGY_NAME="kgeomip"`, $k=3$, y ejecute `uv run exec.py`.
2. **Comparar $k = 2, 3, 4$:** cambie k y vuelva a ejecutar; anote la pérdida de cada uno (crecerá con k).
3. **Comparar estrategias:** repita con `STRATEGY_NAME="kqnodes"` y con `"clustering"` sobre el mismo sistema.

6. Opciones y parámetros avanzados

6.1. Parámetros de configuración

Tabla 2: Parámetros ajustables.

Parámetro	Dónde	Valores	Por defecto
<code>k</code>	<code>main.py</code> / <code>IIT_K</code>	entero ≥ 2 (2–5)	3
<code>STRATEGY_NAME</code>	<code>main.py</code>	<code>kgeomip</code> , <code>kqnodes</code> , <code>clustering</code> , <code>exhaustive</code> , <code>genetic</code> , <code>annealing</code> , <code>tabu</code>	<code>kgeomip</code>
<code>method</code>	<code>main.py</code> / <code>IIT_METHOD</code>	<code>spectral</code> , <code>kmeans</code> (solo <code>clustering</code>)	<code>spectral</code>
<code>PAGE</code>	<code>main.py</code>	A, B, C, ...	A
<code>initial_state</code>	<code>main.py</code>	cadena de 0/1 de longitud n	—
<code>semilla</code>	<code>application.py</code>	entero	73

6.2. Modos de operación

- **Individual** (`uv run exec.py`): un sistema, configurado en `main.py`.
- **Por lotes** (`uv run exec.py -batch`): procesa una rejilla desde Excel, controlado por variables de entorno:

```
1 IIT_STRATEGY=kqnodes IIT_K=4 IIT_METHOD=spectral uv run exec.py --batch
```

- **Exacto** (`STRATEGY_NAME="exhaustive"`): *ground truth* por enumeración; úselo solo en sistemas pequeños ($n \leq 6$).

6.3. Opciones de salida y visualización

```

1 uv run scripts/run_benchmark.py          # rejilla -> CSV/Excel en data/results/
2 uv run scripts/run_benchmark.py --no-meta # sin metaheurísticas (mas rapido)
3 uv run scripts/make_figures.py           # graficas estaticas (PNG): escalabilidad/
    perdida
4 uv run scripts/make_interactive.py        # graficas interactivas (HTML, Plotly)
5 uv run scripts/make_viz.py --net N4A --k 3 --strategy KGeoMIP # dibuja la
    particion

```

Las gráficas **estáticas** (PNG) quedan en `data/results/figures/` y las **interactivas** (HTML, navegables con *zoom* y *hover*) en `data/results/figures/interactive/`.

6.4. Optimización de rendimiento

- Para sistemas grandes, prefiera `kgeomip` (más rápido que `kqnodes`).
- Desactive el *profiling* para medir tiempos reales (`application.disable_profiling()`).
- El paralelismo por procesos se activa en las rutas que lo soportan; evite sobre-suscribir núcleos (el software ya fija `OMP_NUM_THREADS=1` en los trabajadores).

7. Interfaz web (sin línea de comandos)

Para quien prefiera no tocar `main.py`, el proyecto incluye una **interfaz web** (Streamlit) que cubre todo el flujo desde el navegador. Requiere los extras opcionales:

```

1 uv sync --extra web          # instala streamlit + plotly
2 uv run streamlit run app/streamlit_app.py

```

Se abre en `http://localhost:8501`. La interfaz tiene tres pasos:

1. **Datos.** Elija una muestra existente o *genere una TPM nueva* (nodos n , determinista o continua, semilla) con un botón; el CSV se crea en `data/samples/`.
2. **Estrategia.** Seleccione la estrategia (KGeoMIP, KQNodes, Clustering, Genética, Recocido, Tabú, ExhaustiveK), el número de bloques k y, si lo necesita, las máscaras de subsistema (condición, *purview*, mecanismo).
3. **Resultados.** Al pulsar *Ejecutar* verá la pérdida δ_k , la k -partición de mínima información, su distribución marginal en tabla, y el diagrama **interactivo** de la partición. Si existe la rejilla de *benchmark*, se muestra además la gráfica interactiva δ_k vs k .

Sugerencia de captura: para el manual/vídeo, capture (a) el panel de generación de TPM, (b) la selección de estrategia y k , y (c) el panel de resultados con el diagrama interactivo.

8. Solución de problemas

Tabla 3: Errores comunes y solución.

Síntoma	Causa	Solución
FileNotFoundError: N{n}-{page}.csv	No existe la muestra para ese n /página	Generar la red o ajustar PAGE/initial_state
ValueError: k must be ≥ 2	$k < 2$	Usar $k \geq 2$
ValueError: strict k-partition ...	La partición dejaría un bloque vacío	Reducir k (no caben tantos bloques no vacíos)
MemoryError / proceso “muerto”	n demasiado grande (RAM)	Bajar n , o usar clustering
ModuleNotFoundError	Dependencias no instaladas	Ejecutar <code>uv sync</code>
ModuleNotFoundError: streamlit / plotly	Faltan los extras de la UI web	Ejecutar <code>uv sync -extra web</code>
Comando uv no encontrado	uv no instalado/PATH	Reinstalar uv y reabrir la terminal

Problemas de instalación. Si `uv sync` falla, verifique la conexión y la versión de Python (`uv python install 3.14`). En Windows, ejecute la terminal como usuario normal (no se requieren permisos de administrador).

Datos de entrada problemáticos. La TPM debe tener exactamente 2^n filas y n columnas con solo 0/1. Las cadenas `initial_state/conditions/purview/mechanism` deben tener longitud n .

9. Ejemplos y tutoriales

9.1. Tutorial básico (4 nodos)

1. En `main.py`: `initial_state="1111", conditions="1111", purview="1111", mechanism="1111", k=3, STRATEGY_NAME="kgeomip", PAGE="A"`.
2. Ejecutar `uv run exec.py`.
3. Leer la pérdida y los bloques. Dibujar la partición: `uv run scripts/make_viz.py --net N4A --k 3 --strategy KGeoMIP`.

9.2. Caso intermedio (10 nodos, varios k)

Con la red N10A, ejecute KGeoMIP para $k = 2, 3, 4$ y compare la pérdida (debe crecer): $0,47 \rightarrow 0,94 \rightarrow 1,42$. Repita con `kqnodes` para confirmar que obtiene la misma calidad.

9.3. Ejemplo avanzado (rejilla completa)

```

1 uv run scripts/run_benchmark.py           # genera la rejilla n x k
2 uv run scripts/validate_correctness.py     # valida contra el exacto
3 uv run scripts/make_figures.py             # produce las graficas

```

Los resultados quedan en `data/results/` (CSV/Excel) y las figuras en `data/results/figures/`.

10. Referencia rápida

10.1. Comandos principales

Comando	Descripción
<code>uv sync</code>	Instalar dependencias
<code>uv run exec.py</code>	Análisis individual (config. en <code>main.py</code>)
<code>uv run exec.py --batch</code>	Análisis por lotes (Excel)
<code>uv run pytest</code>	Ejecutar los tests
<code>uv run scripts/run_benchmark.py</code>	Rejilla de resultados
<code>uv run scripts/make_viz.py</code>	Visualizar una partición

10.2. Glosario

TPM Matriz de probabilidad de transición que describe el sistema.

k -partición División del sistema en k bloques no vacíos.

Pérdida (φ / δ_k) Información destruida por una partición; menor es mejor.

KGeoMIP Estrategia geométrica (rápida, recomendada por defecto).

KQNodes Estrategia submodular (más robusta, más lenta).

Clustering Baseline rápido para sistemas grandes (solo propone).

Purview / Mechanism Conjuntos de nodos de efecto ($t+1$) y mecanismo (t) bajo análisis.

10.3. Formato de archivos

- **Entrada:** `data/samples/N{n}-{página}.csv` — 2^n filas, n columnas, 0/1, little-endian.
- **Salida:** resumen por consola; rejillas en `data/results/*.csv/*.xlsx`; figuras PNG en `data/results/figures/`.